

Goose + Ollama + GOWA

A Clean Setup Guide for a Local AI Agent Stack on WSL2

Harikumar SV

September 14, 2026

Abstract

This guide walks a new user from zero to a working local AI agent stack on Ubuntu/WSL2 in about 20 minutes. It uses **Ollama** for the local LLM, **Goose** as the agent, **GOWA** for WhatsApp integration via MCP, and optionally **Tavily** for web search.

Contents

1	Prerequisites	2
2	Step 1 — Install Goose	2
3	Step 2 — Install Ollama and Pull a Tool-Capable Model	2
4	Step 3 — Start Ollama with a Larger Context Window	3
5	Step 4 — Configure Goose with Ollama	3
6	Step 5 — Set Up GOWA (WhatsApp MCP)	3
7	Step 6 — Verify the MCP Endpoint	4
7.1	Step 6a — Generate the Base64 String	4
7.2	Step 6b — Save it to a Shell Variable	4
7.3	Step 6c — Test the MCP Endpoint	5
8	Step 7 — Add GOWA to Goose	5
9	Step 8 — Add Tavily (Optional — Web Search)	5
10	Critical Gotchas	6
11	The Two Rules That Save the Most Time	6
12	Working Reference	6
13	Final Status	7

1 Prerequisites

- Ubuntu running on WSL2 (or native Linux)
- Docker and Docker Compose installed
- Node.js and **npm** installed (for the optional Tavily extension)
- A working internet connection for the initial downloads
- A smartphone with WhatsApp installed (for the GOWA section)

Quick check:

```
docker --version
docker-compose --version
node --version
```

If any of these are missing, install them before proceeding.

2 Step 1 — Install Goose

```
curl -fsSL https://github.com/aaif-goose/goose/releases/download/stable/download_cli.
sh | bash

# Add Goose to PATH if needed
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc

# Install bzip2 if extraction fails
sudo apt install bzip2 -y

# Verify
goose --version
```

3 Step 2 — Install Ollama and Pull a Tool-Capable Model

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Stop any auto-started service (we will run it manually)
sudo systemctl stop ollama 2>/dev/null
pkill ollama 2>/dev/null

# Pull a small, fast, tool-calling model
ollama pull qwen3:1.7b
```

Why qwen3:1.7b?

- ~1.4 GB — loads in seconds on a CPU-only laptop.
- Natively supports **tool calling** — required by Goose.
- Larger models such as **ministral-3:8b** also work, but they have a 30–120 second cold start on CPU that can time out Goose's configuration check.

4 Step 3 — Start Ollama with a Larger Context Window

This step is **essential**. Goose's tool schemas exceed Ollama's default 4k context and will be silently truncated otherwise, causing tool calls to fail.

```
pkill ollama
OLLAMA_CONTEXT_LENGTH=16384 ollama serve
```

Leave this terminal running. Open a **new terminal** for subsequent steps.

Verify in the new terminal

```
curl http://localhost:11434/api/tags
ollama list
```

5 Step 4 — Configure Goose with Ollama

```
goose configure
```

Navigate through the wizard:

- **Configure Providers** → **Ollama**
- API host: `http://localhost:11434` (press Enter for default)
- Model: `qwen3:1.7b`

Verify

```
mkdir ~/goose-test && cd ~/goose-test
goose session
```

Inside the session, ask:

what can you do for me?

You should get a coherent response within a few seconds.

6 Step 5 — Set Up GOWA (WhatsApp MCP)

Create `~/gowa/docker-compose.yml`:

Listing 1: GOWA docker-compose.yml

```
services:
  gowa:
    image: aldinokemal2104/go-whatsapp-web-multidevice:latest
    container_name: gowa
    restart: always
    ports:
      - "3000:3000"
    volumes:
      - gowa_storages:/app/storages
      - gowa_statics:/app/statics
    environment:
```

```
- APP_PORT=3000
- APP_DEBUG=true
- APP_OS=QuillAI
- APP_BASIC_AUTH=admin:changeme
- WHATSAPP_AUTO_MARK_READ=true
- WHATSAPP_AUTO_DOWNLOAD_MEDIA=true
- MCP_ENABLED=true
command:
  - rest

volumes:
  gowa_storages:
  gowa_statics:
```

Start it:

```
cd ~/gowa
docker-compose up -d
docker ps    # should show gowa Up on port 3000
```

Link your WhatsApp

Open <http://localhost:3000> in a browser and scan the QR code from WhatsApp → **Linked Devices**.

7 Step 6 — Verify the MCP Endpoint

7.1 Step 6a — Generate the Base64 String

Your GOWA config uses `APP_BASIC_AUTH=admin:changeme`. The MCP endpoint expects an `Authorization: Basic <base64>` header, where `<base64>` is the encoding of `username:password`.

```
echo -n "admin:changeme" | base64
```

You will get output like:

```
YWRtaW46Y2hhbmdlbWU=
```

That entire string is your `<PASTE_BASE64_HERE>`.

Two Critical Details

- **The `-n` flag matters.** Plain `echo` appends a newline before the pipe, which gets encoded into the Base64. The server then decodes `admin:changeme\n` and rejects it as invalid.
- **The colon is literal.** It is `admin:changeme` with no spaces around the colon.

7.2 Step 6b — Save it to a Shell Variable

```
export GOWA_AUTH=$(echo -n "admin:changeme" | base64)
echo "Authorization header value: Basic $GOWA_AUTH"
```

Now you can reference `$GOWA_AUTH` anywhere in this terminal session.

7.3 Step 6c — Test the MCP Endpoint

```
curl -i -X POST http://localhost:3000/mcp \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -H "Authorization: Basic $GOWA_AUTH" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize",
      "params":{"protocolVersion":"2024-11-05",
                "capabilities":{},
                "clientInfo":{"name":"test","version":"1.0"}}}'
```

Expected outcome: HTTP/1.1 200 OK with a JSON body containing the server name WhatsApp Web Multidevice MCP Server.

If you see that, the server side is fully working. If you get a 401, re-check the Base64 string with `echo -n`.

8 Step 7 — Add GOWA to Goose

Edit `~/.config/goose/config.yaml` and add:

Listing 2: Goose config.yaml — GOWA extension

```
extensions:
  gowa:
    enabled: true
    name: GOWA WhatsApp
    type: streamable_http
    uri: http://localhost:3000/mcp
    timeout: 300
    headers:
      Authorization: Basic YWRtaW46Y2hhbmdlbWU=
```

The value after `Basic` is the exact output from `echo -n "admin:changeme" | base64`. Restart Goose cleanly:

```
pkill goose
goose session
```

Verify by asking inside the session:

What tools do you have from the GOWA extension?

You should see `whatsapp_send_text`, `whatsapp_send_image`, `whatsapp_send_location`, and similar tools.

9 Step 8 — Add Tavily (Optional — Web Search)

1. Get a free API key at <https://tavily.com> (starts with `tvly-`).
2. Ensure Node.js is installed:

```
node --version || sudo apt install nodejs npm -y
```

3. Run `goose configure` → **Add Extension** → **Command-line Extension**:

- Name: Tavily Web Search

- Command: `npx -y tavily-mcp`
- Timeout: 300
- Env var: `TAVILY_API_KEY` → your key

4. Restart Goose and test:

Search the web for the latest WSL2 networking tips.

10 Critical Gotchas

Gotcha	Fix
Ollama's default 4k context breaks tool calling	Always start with <code>OLLAMA_CONTEXT_LENGTH=16384 ollama serve</code>
<code>ollama list</code> empty after pull	Run <code>ollama serve</code> in the user shell, not the systemd service
Model rejects tool calls	Use a model with <code>tools</code> capability: <code>qwen3:1.7b</code> , <code>minstral-3:8b</code> , <code>llama3.3</code>
Goose shows stale tool list	<code>pkill goose</code> and start a fresh session after config changes
"Auth required" from GOWA MCP	Use explicit <code>Authorization: Basic <base64></code> header, not <code>user/password</code> fields
<code>headers</code> format	YAML <code>map</code> (key: value), not a list of name/value objects
<code>echo</code> adds trailing newline to Base64	Use <code>echo -n</code> — otherwise encoded credentials include <code>\n</code> and fail
Port 3000 conflict	Goose's OAuth callback also uses 3000. If GOWA is on 3000, skip OAuth and use Basic Auth
GOWA <code>mcp</code> subcommand error	Newer GOWA versions unify REST + MCP on one port. Use <code>command: rest</code> with <code>MCP_ENABLED=true</code>
First prompt is slow	CPU-only cold start takes 30–120s; subsequent prompts are faster

11 The Two Rules That Save the Most Time

1. Always verify each layer independently before blaming the next.

- `ollama run qwen3:1.7b "hi"` before blaming Goose.
- `curl` the MCP endpoint before blaming Goose's MCP client.
- `curl` the Tavily API directly before blaming the extension.

2. Restart Goose after every config change.

- `pkill goose` then `goose session`.
- Stale caches are the source of most "it's not working" moments.

12 Working Reference

Ollama Terminal (Leave Running)

```
pkill ollama
OLLAMA_CONTEXT_LENGTH=16384 ollama serve
```

Goose Config (~/.config/goose/config.yaml)

```
extensions:
  gowa:
    enabled: true
    name: GOWA WhatsApp
    type: streamable_http
    uri: http://localhost:3000/mcp
    timeout: 300
    headers:
      Authorization: Basic <BASE64_OF_admin:changeme>
providers:
  ollama:
    enabled: true
    model: qwen3:1.7b
    configured: true
OLLAMA_HOST: http://localhost:11434
```

13 Final Status

Working Configuration

The full stack is operational:

Goose (WSL2) → MCP → GOWA (Docker) → WhatsApp
Goose (WSL2) → MCP → Tavily (npx) → Web

All components run locally. No cloud dependency for the WhatsApp leg. The LLM runs locally via Ollama. Web search reaches the internet only through Tavily's API.